teammates might not. In this case, you could put --fail-fast in your ~/.rspec file, and this setting would apply just for you.

*./.rspec*

This file in a project's root directory is for *project-level defaults*. Use restraint here; only put options that are necessary for the suite to run correctly, or for standards that your team agrees upon. For example, if there's a file you always want loaded, you can --require it automatically. (Indeed, when you generate a new project with rspec --init, RSpec puts --require spec_helper into the project's .rspec file for you.)

*./.rspec-local*

This file, which lives alongside a project's .rspec file, is for your *personal preferences* for that project. Since everyone may have their own version of this file, make sure to exclude it from your source control system.

Options take precedence in the order we've listed them here, meaning that local options will override more global ones. For instance, if your project has --profile 5 set in its .rspec file, you could override this setting by putting --no-profile in the project's .rspec-local file.

You can also set command-line defaults in the SPEC_OPTS environment variable; values set here will override ones set in text files.

Before we move on to the next way to configure RSpec, let's take a moment to put this knowledge into practice. We're going to set up RSpec to use a custom formatter to get the exact report output we want.

## Using a Custom Formatter

In *Customizing Your Specs' Output*, on page 16, you used RSpec's built-in formatters to view different levels of detail in your specs' output. Now, you're going to use a *custom formatter* to make one small tweak to the way RSpec reports failures.

A custom formatter is a regular Ruby class that registers itself with RSpec to receive *notifications*. As your suite runs, RSpec notifies the formatter of the events it's subscribed to, such as starting an example group, running an example, or encountering a failure. This flexible system gives you room for all sorts of creative changes to the output. Here, we're going to change how RSpec reports failures.

RSpec's built-in formatters display failure details—messages and back-traces—at the very end of the run. This is a good default, since it puts a handy work list at the end of the output where it's easy to find.